SciLifeLab

BUILDING SECURE CONTAINER IMAGES

DEVOPSDAYS ZURICH

ALVARO REVUELTA

SciLifeLab

ABOUT ME

Alvaro Revuelta.

- Systems Developer
- Laboratory for Life Sciences - SciLifeLab
- Get the code (and slides)

images/qrmeetup.png

5

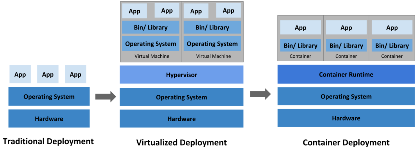
SciLifeLab

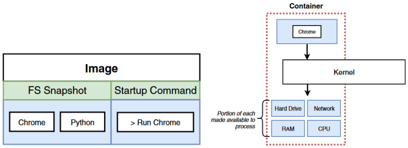
PRESENTATION STRUCTURE

The explanation on the slides is interluded with the explanations in the README in the repository

1. Introduction
2. Image Structure
3. How layers are generated
4. Optimize images
5. Conclusion

1. INTRODUCTION





(a) image

(b) container

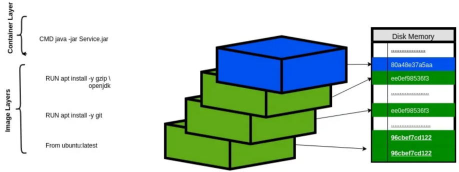
1. INTRODUCTION

- Images are **immutable**. Once an image is created, it can't be modified.

- Container images are composed of **layers**. Each layer represents a set of file system changes that add, remove, or modify files.

2. IMAGE STRUCTURE

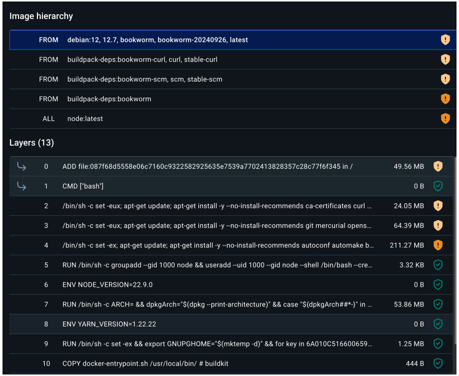
- Images consist of **multiple layers** stacked on top of each other.
- Layers are **file system changes** (e.g., file additions, deletions, updates).
- Layers are **read-only**; only the top container layer is **writable** at runtime.
- Layers are merged to form a **Unified File System** (Full explanation would need another presentation).



(**Source**: UnionFS : A File System of a Container)

3. HOW IMAGES ARE GENERATED

- Layers are **reusable** in subsequent builds, because they are cached.
- **FROM**: Setting the base image (starting point)
- **RUN**: Run a command (e.g., installing software)
- **COPY/ADD**: Adding files
- **CMD/ENTRYPOINT**: Setting the default command for running the container
- RUN, ADD and COPY generate new layers.



(**Source**: DockerHub)

SciLifeLab

4. OPTIMIZE IMAGES

trivy image imagetag:version

```
/ # trivy image rv0lt/flaskrediswebapp:basic | head
2023-04-21T18:04:09.297Z        INFO    Vulnerability scanning is en
2023-04-21T18:04:09.297Z        INFO    Secret scanning is enabled
2023-04-21T18:04:09.297Z        INFO    If your scanning is slow, pl
2023-04-21T18:04:09.297Z        INFO    Please see also https://aqua
ret detection
2023-04-21T18:04:11.456Z        INFO    Detected OS: debian
2023-04-21T18:04:11.456Z        INFO    Detecting Debian vulnerabili
2023-04-21T18:04:11.691Z        INFO    Number of language-specific
2023-04-21T18:04:11.691Z        INFO    Detecting python-pkg vulnera

rv0lt/flaskrediswebapp:basic (debian 10.13)
===========================================
Total: 146 (UNKNOWN: 5, LOW: 86, MEDIUM: 21, HIGH: 32, CRITICAL: 2)
```

(github.com/aquasecurity/trivy)

SciLifeLab

4. OPTIMIZE IMAGES

Minimizing the Number of Layers:

- Combine multiple RUN commands into one (e.g., chaining commands using &&).
- Removing unnecessary files during the build process (e.g., apt-get clean).
- Avoid using COPY to add files that won't be needed during runtime.

Using Smaller Base Images:

- Alpine Linux vs. Ubuntu: Trade-offs between size and functionality.
- Multi-stage builds: Using one image for building the application and another for the final image (stripped of development tools).

Caching Layers Efficiently:

- Take advantage of layer caching by ordering Dockerfile instructions logically.
- Place less frequently changed commands earlier in the Dockerfile to maximize caching.

Removing Unnecessary Packages and Files:

- Remove temporary files, build dependencies, or logs to reduce image size.
- Clean up the file system after installations (e.g., rm -rf /var/lib/apt/lists/*).

SciLifeLab

4. OPTIMIZE IMAGES

Subsequent building times: +60s

Dockerfile basic | Dockerfile basic2 | Do

basic > Dockerfile > ...

```
# use ubuntu as base image
FROM ubuntu

# copy the source code
COPY hello.c hello.c

# install build-essential package to compi
RUN apt update
RUN apt install -y build-essential

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# Run the program
ENTRYPOINT ["/helloWorld"]
```

Subsequent building times: 1s

Dockerfile basic | Dockerfile basic2 | Doc

basic2 > Dockerfile > ...

```
# use ubuntu as base image
FROM ubuntu

# install build-essential package to compil
RUN apt update
RUN apt install -y build-essential

# copy the source code
COPY hello.c hello.c

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# Run the program
ENTRYPOINT ["/helloWorld"]
```

4. OPTIMIZE IMAGES

```
# use ubuntu as base image
FROM ubuntu as build-env

# install build-essential package to compile the source
RUN apt update && apt install -y build-essential

# copy the source code
COPY hello.c hello.c

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# FROM alpine for an even greater size reduce
FROM ubuntu

# copy binary executable to new layer
COPY --from=build-env ./helloWorld ./helloWorld

# Run the program
ENTRYPOINT ["/helloWorld"]
```

WEIGHT IN MB REDUCED BY 4X TIMES

4. OPTIMIZE IMAGES - DISTROLESS

- If every change in the file system creates a new layer. Can there be an empty image?
- Yes! Introducing **FROM SCRATCH**
- Images built using this base are called **distroless**.
- By reducing the image size, you are also reducing the **attack surface** for possible vulnerabilities.

4. OPTIMIZE IMAGES - DISTROLESS

- Even thought there is a considerable increase in security and performance, images built this way introduce a considerable **development overhead**.
- For learning purposes they are pretty useful, but for production ready environment maybe not the best.
- Google realized this some years back and developed their own **suite of distroless images** (link to Google Distroless repository).

```
# use ubuntu as base image
FROM ubuntu as build-env

# install build-essential package to compile the source code
RUN apt update && apt install -y build-essential

# copy the source code
COPY hello.c hello.c

# Compile and generate binary
RUN gcc -o helloWorld hello.c

# use distroless container to run the program
FROM SCRATCH

# copy the required libraries to run the program
COPY --from=build-env  /lib/aarch64-linux-gnu/libc.so.6 /lib/aarch64-linux-gnu/libc.so.6
COPY --from=build-env  /lib/ld-linux-aarch64.so.1 /lib/ld-linux-aarch64.so.1

# copy the /etc/passwd
COPY --from=build-env  /etc/passwd /etc/passwd

# tmp directory
COPY --from=build-env  /tmp /tmp

# copy binary executable to new layer
COPY --from=build-env ./helloWorld ./helloWorld
```

(**Source**: own)

SciLifeLab

4. OPTIMIZE IMAGES - DISTROLESS

- In the quest for optimizing images, some projects have appeared.
- To debug them in platform such as Kubernetes, one can use ephemeral containers.

- Ubuntu Chiselled images (link).
- COPA project (link).

5. CONCLUSSION

Key Takeaways

- Importance of understanding image layers and their role in optimization.
- Best practices for reducing image size and improving efficiency.
- Tools and techniques for analyzing and improving images.

Q&A ?